

Movie Night Application Report

Elisa Ciardulli

Application Architecture

I designed the architecture of my application around the main architecture principles: separation of concerns, generating UI based on the data models, single source of truth and unidirectional data flow.

Data Layer

The application uses an *AppContainer* to manage all the app's data sources (separation of concerns), be it data from the Internet or locally stored. It defines the Retrofit instance for network requests and the Room database for local storage. By holding these repositories in one place, it ensures the rest of the app gets its data from a consistent source.

Repositories

By using repositories, the UI layer interacts with a Single Source of Truth without needing to know the underlying implementation details. The rest of the app can just interact with these simple interfaces to get the necessary data from both remote and local sources. There are two repository interfaces: the *MoviesRepository* implemented by *NetworkMoviesRepository* handles remote API calls, while the *SavedMoviesRepository* implemented by the *OfflineSavedMoviesRepository* manages the local database via Room.

Data Sources

The Remote Data Source (REST API) is handled by the *MoviesApiService*, which is responsible for making requests to the KinoCheck API.

The Local Data Source (Room Database) is handled by the *SavedMoviesDAO*, which provides the actual implementation for SQLite commands (Insert, Delete, Query, etc.) to interact with local storage using the *MovieToWatch* and *WatchedMovie* entities.

Domain Layer

Since the application does not require complex business logic that needs to be reused by multiple ViewModels, all logic is included directly in single ViewModels.

Ui Layer

The UI Layer is composed of main screens and reusable components, which are driven by the ViewModels that manage the UI states, making sure there is a unidirectional data flow between the data sources and the user interface.

The user interacts with the screens, which send the Events down to the appropriate ViewModels (separated by concerns), which process it and change the UI State accordingly. The changes to the UI then travel back up to the screen to change its appearance.

UI Elements

There are four main screens:

- *HomeScreen*: contains a list of movies divided by genre and the watchlist. Uses the *MovieDetails* component to show the movie details of a selected movie. Also contains the Movie Night button to start a movie night event.
- *AddFriendsScreen*: contains a list of *Friend* objects which are represented through random colourful icons and the start button to start the voting process. It also displays a dialog box to enter the name of new participants.
- *VoteScreen*: displays the current *Friend* voting, the picture of the movie, a dialog box at the start of each round, the voting buttons and the *MovieNightEventNavBar*, as well as displaying the details of a movie with the *MovieDetails* component, when a movie is selected.
- *RankingList Screen*: Displays a list of cards representing the movies and also uses the *MovieNightEventNavBar* and *MovieDetails* composables.

I defined some composables in a separate package, because they are reused in several screens: for example the *ThemeToggleButton*, which is used in every screen.

State Holders

There are several ViewModels in this application, thus maintaining the separation of concerns:

- *ThemeViewModel*: used in all screens to toggle the light/dark theme
- *HomeViewModel*: only used in the *HomeScreen* to handle watchlist
- *MovieViewModel*: handles the movie lists divided by genre in the *HomeScreen*
- *MovieDetailsViewModel*: used in all screens except the *AddFriendsScreen* to handle the showing and hiding of the details of a selected/deselected. Also handles the “To Watch” and “Watched” toggles.
- *MovieNightEventViewModel*: handles the entirety of a movie night event (*AddFriendsScreen*, *VoteScreen*, *RankingListScreen*) by handling the start and end of the event, keeping track of the participants (that can be removed and added), the list of movies to vote on, the going to the next movie or the next person and finally displaying the final ranking logically.

Each of these ViewModels manages their own UI state, which serves as a read-only resource based on which the app decides what to display.

The *AppViewModelProvider* is responsible for creating each ViewModel when it is actually needed by a screen. It uses the *AppContainer* to inject the right repositories into the ViewModels during this setup.

Application Design

The design of my application started with the build of the Material Theme through the Material Theme Builder online resource. I then checked that both light and dark themes were accessible and usable.

The Material design includes typography (consistent fonts and sizes of text), shape (how to handle shapes of different sizes), together with theme and colors defining the main color palette of the application.

The design of the application works well for both compact and expanded window sizes, but I did implement a couple of layout differences based on the window size:

The *MovieDetails* are shown in a dialog card when in compact mode and in a side panel when in expanded mode. This layout is defined by checking which is the current *MovieNightContentType*, either COMPACT for compact, or EXPANDED for expanded window sizes.

Moreover in some pages, the size of the movie pictures and of the text are defined based on the size of the window.

Navigation

The navigation is handled almost entirely in the *MovieNightApp*.

- From the *HomeScreen* navigate to the *AddFriendsScreen* by clicking the Movie night button.
- From the *AddFriendsScreen* either navigate back to the *HomeScreen* using the back button or continue to the *VoteScreen* by clicking the Start button (only if there were no errors in the start of the movie night event)
- Both the *VoteScreen* and the *RankingListScreen* contain a *NavBar* that allows you to either go back to the *HomeScreen* or “Try Again” and start a new movie night event starting from a fresh *AddFriendsScreen*.

The navigation through the system navigation bar is also handled to avoid unwanted user behavior:

- Whenever a movie night event is completed, all *VoteScreens* and the *AddFriendsScreen* are removed from the stack, so going back will lead directly to the *HomeScreen*. This is to avoid meddling with a complete event and allow the user to start over completely.
- In the *VoteScreen* trying to go back resets the movie night event and goes back to the *AddFriendsScreen* with a fresh start, in order to not have old data in the new event.

REST API

The app uses the Retrofit library to fetch movie data from the KinoCheck REST API. The *AppDataContainer* parses the responses into the data models: *Movies* and *MovieDetails*. This remote data source is hidden behind *NetworkMoviesRepository*, which acts as a single interface for the rest of the app to fetch movies or movie details.

Internal Database

The app uses a Room database to save the data locally. This keeps the data safe and accessible even when offline. The entities used in the app are *MovieToWatch* and *WatchedMovie* which both contain basically a simple *Movie* object and define the schema for the two tables: *watchedMovies* and *moviesToWatch*.

The *SavedMoviesDAO* provides the interface for all operations to create, delete, get etc. instances from local tables, using Flow to provide updates to the UI as soon as the database changes.

The *SavedMoviesDatabase* provides a Singleton instance of the database, making sure the entire app uses the same copy of the database, providing consistency all around.

Services or WorkManager

My application does not contain Services or WorkManagers, this was due to time constraints. It may have been useful to implement a WorkManager called *UploadWorker* to upload and save in the local database the movies for the home page every two hours, through a *PeriodicWorkRequest*, to make the loading of the home page upon opening the app, quicker and smoother.